

Document de passation — Plateforme PASS (ANSUT)

Programme d'Accès aux Smartphones Subventionnés · à l'attention de l'équipe de développement

Nature du livrable. Prototype fonctionnel de démonstration, complet côté parcours métier et règles, servant de **référence illustrative** au passage en production. Les vérifications aux référentiels nationaux sont **simulées** ; l'intégrité métier, elle, est **réelle et effective en base**.

1. Accès et liens

Application en ligne	https://somet1010.github.io/pass-prototype/
Dépôt	https://github.com/SOMET1010/pass-prototype
Projet Supabase (réf.)	gkxmopdyudfuadhxyuwe — région à confirmer
Hébergement front	GitHub Pages (workflow .github/workflows/deploy.yml)

Comptes de démonstration — mot de passe commun [passdemo2026](#)

Rôle	E-mail	Portée
Enrôlement	enrolement@pass.demo	Enrôlement
Instructeur	instructeur@pass.demo	Vérification / décision
Remise	remise@pass.demo	Remise des terminaux
Superviseur	superviseur@pass.demo	Supervision + logistique + paramètres + ciblage géo + annulations

2. Stack technique

- **Front** : React 18 + TypeScript + Vite, React Router, Tailwind (classes utilitaires), [lucide-react](#) (icônes), [leaflet](#) (carte), [qrcode](#) . SPA authentifiée (4 rôles).
- **Back / données** : Supabase = **PostgreSQL** (RLS, fonctions [SECURITY DEFINER](#) , triggers, vues) + Auth (OIDC e-mail/mot de passe) + Edge Functions (Deno) + Storage (non utilisé de façon critique dans le prototype).
- **Intégrations** : 2 Edge Functions ([ansut-hub](#) messagerie, [ansut-cachet](#) cryptologie).
- **CI/CD** : GitHub Actions → build Bun/Vite → GitHub Pages.

Correspondance avec l'architecture cible Azure : voir [CONFORMITE_ARCHITECTURE.md](#) . Le socle données est en **SQL standard**, portable vers **Azure Database for PostgreSQL** sans réécriture.

3. Configuration & secrets

Front (variables de build Vite)

Variable	Rôle
VITE_SUPABASE_URL	URL du projet Supabase
VITE_SUPABASE_ANON_KEY	Clé publique (anon) — non secrète

Définies dans `deploy.yml` (valeurs de démo en repli) ou en *GitHub* → *Settings* → *Secrets and variables* → *Actions*.
En local : fichier `.env` (voir `.env.example`).

Edge Functions (secrets Supabase — *Project Settings* → *Edge Functions* → *Secrets*)

Secret	Fonction	Rôle
ANSUT_HUB_URL / ANSUT_HUB_USERNAME / ANSUT_HUB_PASSWORD	ansut-hub	Passerelle messagerie (SMS/e-mail/WhatsApp)
ANSUT_CRYPTO_URL / ANSUT_CRYPTO_USERNAME / ANSUT_CRYPTO_PASSWORD / ANSUT_CRYPTO_AUTORITE	ansut-cachet	Cryptologie (signature qualifiée)
SUPABASE_URL / SUPABASE_ANON_KEY / SUPABASE_SERVICE_ROLE_KEY	les deux	Injectés automatiquement par Supabase

En l'absence de ces secrets, les fonctions basculent en mode « simulé » sans bloquer le parcours.
Les fournir suffit à passer en réel (aucun code à changer).

4. Lancer en local

```
git clone https://github.com/SOMET1010/pass-prototype.git
cd pass-prototype
cp .env.example .env      # renseigner VITE_SUPABASE_URL et VITE_SUPABASE_ANON_KEY
npm install               # ou bun install
npm run dev               # http://localhost:5173
```

Migrations : appliquer dans l'ordre chronologique les fichiers de `supabase/migrations/` (via Supabase CLI `supabase db push` , ou l'éditeur SQL). Elles sont **idempotentes-friendly** et **rejouables**. Les Edge Functions se déploient avec `supabase functions deploy ansut-hub` et `ansut-cachet` .

5. Structure du dépôt

```
src/
  pages/          20 écrans (parcours + admin)
  components/     Layout, EvaluationV3, Defendabilite, ProfilUsage, CachetPanel, cartes, badges...
  lib/            supabase.ts, types.ts (miroir du schéma), rules.ts, eligibilite.ts,
```

```
defendabilite.ts, logistique.ts, ansut.ts, operateurs.ts, zones.ts, image.ts
supabase/
  migrations/ 29 migrations (schéma + règles + RPC + seeds)
  functions/  ansut-hub, ansut-cachet (+ README)
docs/        CDC v3, Architecture cible, présentation CA, notes de conformité, ce document
.github/workflows/deploy.yml
```

Écrans (`src/pages`)

Login · Accueil · Enrolement · Instruction · Verification (fiche dossier + Éligibilité v3 + Défendabilité) · FicheDossier ·
Dossiers · Recherche · Avis · Convocation · Remise · Recu (+ profil d'usage) · Stock · Logistique · CiblageGeo ·
Simulateur · Parametres · Supervision · Sav · APropos.

6. Base de données

Principes structurants (à préserver) : - **Intégrité en base** : contraintes + triggers + RLS. Toute écriture sensible passe par une **fonction SECURITY DEFINER nommée** (`pass_*`), jamais par un INSERT/UPDATE direct depuis le front. - **RLS** : lecture réservée aux agents actifs (`is_active_agent()`), écriture via RPC contrôlant le rôle (`current_agent_role()`). - **Journal d'audit inaltérable** (`journal_audit` , `_log(...)`), **décision irréversible, snapshots immuables**. - **Paramètres administrables** : aucune valeur métier (poids, seuils) en dur — tout dans `parametre` / `parametre_version` (versionné, horodaté, attribué).

Tables par domaine

- **Parcours & identité** : `agent` , `personne` , `campagne` , `demande` , `verification` , `decision` , `cloture` , `distribution` , `preuve_remise` , `notification` , `journal_audit` .
- **Terminaux & points** : `terminal` , `point_retrait` , `sav_ticket` .
- **Éligibilité v3** : `parametre` , `parametre_version` , `evaluation_individuelle` , `controle_regularity` , `score_dimension` , `source_externes` .
- **Ciblage géographique** : `localite` , `ciblage_geo` , `ciblage_localite` , `arbitrage_reserve` .
- **Logistique** : `fournisseur` , `entrepot` , `emplacement` , `arrivage` , `colisage_ligne` , `colis` , `equipe` , `equipe_membre` , `mission` , `rapport_mission` , `bon_transfert` , `mouvement_stock` , `commande` , `commande_ligne` .
- **Défendabilité** : `decision_snapshot` , `annulation` .
- **Intégrations** : `cachet_electronique` .
- **Base séparée (schéma `app`)** : `app.profil_usage` — non exposée par l'API REST, accès par fonctions uniquement.

Vues

`v_parametres` , `v_stock_points` , `v_stock_logistique` , `v_vitesse_remise` .

RPC principales (`pass_*`)

- **Parcours** : `pass_creer_personne` , `pass_creer_demande` , `pass_enroler` , `pass_enregistrer_consentement` , `pass_lancer_verifications` , `pass_soumettre_demande` , `pass_mettre_en_instruction` , `pass_prononcer_decision` , `pass_effectuer_remise` , `pass_activer_terminal` , `pass_cloturer` .
- **Éligibilité v3** : `pass_evaluer_demande` , `pass_simuler_eligibilite` , `pass_param_maj` .
- **Ciblage géo** : `pass_calculer_ciblage_geo` , `pass_arbitrer_reserve` .

- Défendabilité : `pass_annuler_decision` (+ snapshot par trigger).
 - Logistique : `pass_creer_fournisseur/entrepot/emplacement/arrivee` , `pass_receptionner_colis` , `pass_ranger_colis` , `pass_transferer_vers_point` , `pass_ouvrir_point_mobile` , `pass_config_equipe` , `pass_maj_geoloc` , `pass_demarrer_mission` , `pass_cloturer_mission` , `pass_transferer_stock` .
 - Commandes : `pass_creer_commande` , `pass_maj_statut_commande` .
 - Sources / profil : `pass_source_maj` , `pass_profil_usage_maj` , `pass_profil_usage_lire` .
 - SAV / notifications : `pass_ouvrir_sav` , `pass_traiter_sav` , `pass_notifier_sms` , `pass_notification_maj_dispatch` , `pass_sceller` , `pass_cachet_maj` .
-

7. Modules fonctionnels

1. **Parcours** : enrôlement assisté → vérification → décision → remise → reçu → supervision. Machine à états via `demande.etat` .
 2. **Éligibilité v3 (CDC v3)** : contrôles de **régularité** (bloquants) séparés du **score C1-C5** → **rang P1-P4** ; sortie *recevable / refus / à instruire*. Paramètres administrables, **snapshot de reproductibilité**.
 3. **Ciblage géographique (CDC §2)** : référentiel `localite` , 3 filtres, score 5 critères, quotas (population éligible = population × taux de pauvreté), **réserve d'arbitrage** (motif+auteur+horodatage).
 4. **Logistique** : commande fournisseur → arrivee/colisage → réception (scan) → entrepôt → transfert → point fixe/mobile → équipes → mission → rapport ; **traçabilité IMEI**, journal des mouvements.
 5. **Intégrations ANSUT** : `ansut-hub` (notifications réelles) + `ansut-cachet` (cachet SHA-256 + signature).
 6. **Défendabilité (Architecture §3)** : snapshot de décision immuable + **annulation append-only** (compensation) + quota atomique `FOR UPDATE` + séparation des tâches.
 7. **Simulateur** : `pass_simuler_eligibilite` — calcul pur, sans persistance.
 8. **Profil d'usage (CDC §3.4)** : base séparée, n'alimente jamais l'éligibilité.
-

8. Conformité (état)

- **CDC Éligibilité v3** : ☒ paramètres administrables, régularité/score séparés (P1-P4), reproductibilité, ciblage géographique, profil séparé, modes d'accès sources. Détail dans le CDC archivé (`docs/PASS_CDC_Eligibilite_v3.pdf`).
 - **Architecture cible ANSUT** : ☒ patterns de défendabilité en base (portables Azure) ; ☒ infra/juridique de production (voir §9). Détail dans `CONFORMITE_ARCHITECTURE.md` .
-

9. Reste à faire — feuille de route production

Bloquants organisationnels / juridiques (hors code)

- **ARTCI** : autorisation de **traitement** + autorisation de **transfert hors CI** (loi n°2013-450) ; **DPIA** avant développement.
- **ONECI** : convention d'interconnexion, interface réelle, SLA (conditionne tout le mode dégradé).
- **Région Azure**, souscription, landing zone ; choix **Entra ID vs Keycloak**.

Technique (à industrialiser)

1. **Remplacer les simulations par les vraies sources** (ONECI, RSU, opérateurs, IMEI) derrière l'abstraction `source_externe` (modes fichier/service/déclaratif déjà en place) + **circuit breaker** + cache + figement de

la réponse dans le snapshot.

2. **Migrer l'hébergement** vers Azure : front (App Service/AKS), **Azure Database for PostgreSQL** (les migrations SQL sont portables), **Blob Storage + Key Vault** pour les pièces (CNI, signatures, reçus) avec chiffrement + purge, **Service Bus + pattern Outbox** pour les notifications.
 3. **Identité** : Keycloak/Entra ID (RBAC, MFA, séparation des tâches déjà amorcée en base), fédération gouvernementale éventuelle.
 4. **PWA offline-first** : service worker + file de synchronisation locale + résolution de conflits à la synchro (unicité CNI, quota) — voir CDC/architecture §7.
 5. **Double-contrôle 4-yeux** complet à la remise (au-delà de la séparation des tâches déjà en base).
 6. **Qualité** : suite de **tests** (unitaires règles TS + tests SQL des invariants), **CI/CD** (dev/recette/prod), **IaC** (Bicep/Terraform), observabilité (logs/alerting), sauvegardes testées, RPO/RTO/PRA.
 7. **Sécurité** : revue RLS, rotation des secrets (Key Vault), journal d'accès orienté personne concernée (droit d'accès).
 8. **Valeurs « non arrêtées »** : les paramètres marqués *non arrêté* (poids C1-C5, seuils P1-P4, clé score/réserve 75/25 vs 80/20, non-cumul, ancienneté ligne, seuil d'inactivité...) doivent être **arrêtés par la gouvernance** (décisions C-1 à C-8) puis figés via la page Paramètres.
-

10. Points d'attention (dette / conventions)

- Le prototype utilise **Supabase** (Postgres managé) ; la logique métier étant du SQL standard, la bascule vers Azure PostgreSQL est directe, **mais** l'Auth Supabase (OIDC) devra être remplacée par Entra ID/Keycloak — impact sur `src/lib/supabase.ts` et le contexte d'auth.
 - Les **signaux du score C1-C5** sont, en démo, **dérivés de façon déterministe** (hash) faute de données réelles : à remplacer par les vraies données opérateurs (dont la **date de dernière activité**, déjà prévue dans `source_externes`).
 - Le **mot de passe de démo** et les comptes `@pass.demo` sont à supprimer avant production.
 - La clé **anon** dans `deploy.yml` est publique (normal pour Supabase) ; la **service_role** ne doit jamais être exposée côté front (uniquement Edge Functions).
 - Tout ce qui est **badge « SIMULÉ »** matérialise une donnée non raccordée à un référentiel réel — à lever source par source.
-

11. Documents de référence (dossier `docs/`)

- `PASS_CDC_Eligibilite_v3.pdf` / `.docx` — cahier des charges éligibilité v3.
- `Architecture_PASS_ANSUT.pdf` — architecture cible Azure.
- `CONFORMITE_ARCHITECTURE.md` — mapping exigences ↔ implémentation.
- `PASS_Presentation_Conseil_Administration.pptx` + `PASS_texte_presentation.txt` — support CA.
- `README.md` — index docs + comptes de démo.